

CST4011 Software Development Group Project

Centralized Bug Reporting Platform

Kevin Panchani-M01082687

Module Leader: Elisabeth Fink

Submission Date: 12/04/2026

Contents

1. Introduction and Motivation 4

2. User Stories and Workflow models 5

3. User Interface Design 7

3.1 Registration and Login Interface 7

3.2 User Dashboard 8

3.3 submit Bug 8

3.4 Company Dashboard 10

3.5 Bug Review and Decision Interface 10

3.6 Admin Dashboard 11

4. Code Architecture and Implementation Structure 13

4.1 Tools and language used 13

4.2 Reasons for Choosing These Tools 14

4.3 Code Structure and Workflow Models 15

5. Implementation, Testing and Validation Strategy 18

6. Video Description 19

List of Figure

Fig 1: Use case Diagram 6

Fig 2: Login and Register page 7

Fig 3: User Dahsboard 8

Fig 4: Submit Bug 9

Fig 5: Company dashboard 10

Fig 6: Bug Review 10

Fig 7: Admin Dashboard 11

Abstract

The Bug Reporting Platform refers to a web-based application whose main purpose is to help in the reporting, monitoring, and fixing of bugs found in software programs. It contains separate user types that perform particular activities for their roles: users, company representatives, and an admin. Users are allowed to create accounts, log into the application, and report any bug found on their software programs. Companies are also allowed to create accounts. When they are registered and accepted by the admin, they can monitor the bugs submitted by the user.

If the bug is found to be valid, it means the company will have to accept the submission and award some points to the submitting user. However, if the bug is not valid, it is classified as not approved.

1. Introduction and Motivation

Software development is increasingly becoming a complex process in today's modern times. Defects in software pose problems for both the users and the developers. Apart from the negative impact that bugs, defects, and faults in software have on its performance and effectiveness, they might cause loss of money, security threats, and a bad image for the company. Traditionally, people use informal channels such as e-mails or spreadsheets to report their problems regarding some software or applications.

The Bug Reporting Platform was created to solve this problem. It provides a systematic method for the submission, review, and resolution of bug reports. The Bug Reporting Platform helps bridge the communication gap between software end-users and developers. The motivation for end-users to submit genuine bug reports lies in the rewards they get from their contribution while the company gets to solve defects as early as possible.

The platform features a strong workflow involving three parties: users, companies, and the administrator. The users get the opportunity to report any bug they find while using the software, whereas the companies will be responsible for validating and dealing with such reports. Finally, the administrator will keep track of everything happening on the platform by approving the company members. In this way, the workflow will promote accountability, transparency, and incentivization, thereby ensuring better software quality and higher customer satisfaction.

The main rationale for undertaking the present project was to establish a direct channel of communication between users and software developers.

2. User Stories and Workflow models

User stories describe the requirements of end users in agile development. User stories describe what is needed, by whom, and why. User stories in this project include Bug Reporter (User), Company, and Admin. Each user story is uniquely identified. Each user story aims at delivering value with traceability. Each user story is aligned with system goals.

Bug Reporter (User)	**Register & Onboard**: As a bug reporter, I want instant registration without admin approval so I can start reporting bugs immediately. **Submit Bug Report**: As a bug reporter, I want to submit a bug with website name, URL, description, and supporting media so the company can reproduce it. **Track Bug Status**: As a bug reporter, I want a dashboard showing all submitted bugs with their status (Pending, Approved, Rejected) for easy tracking. **View Reward Wallet**: As a bug reporter, I want to see my total reward points and points per bug to stay motivated.
Company	**Registration and Pending Verification**: As an organization, I want to register and wait for admin approval to ensure only genuine organizations are verified. **Review Incoming Bug Reports**: As an organization, I want my dashboard to list all bug reports with details and media for efficient review. **Approve or Reject Bug**: As an organization, I want to mark a bug as Approved or Rejected and provide optional feedback to users. **Allocate Reward Points**: As an organization, I want to assign reward points based on the severity of approved bugs (Low, Medium, Critical).
Admin	**Approve or Reject Company Registration**: As an admin, I review company registration requests to certify legitimate organizations. **Monitor All Bug Reports**: As an admin, I want an overview of all bug reports platform-wide to detect spam or suspicious activity. **Manage Users & Companies**: As an admin, I want to suspend, reactivate, or delete accounts to enforce policies and prevent abuse. **Platform Analytics Overview**: As an admin, I need summaries of submitted, approved, and rejected bugs, and total active users and companies.



Fig 1: use case diagram

Use Case diagram shown below represents the requirements of Centralized Bug Reporting system in terms of functional aspects where all interactions are taking place among the three actors that include User (Bug Reporter), Company, and Admin. Process starts from Bug Reporter that should be able to register on the platform, login into the account, submit a bug report and then track the status of his/her report and accumulated reward points. As far as Company goes, it deals with view, approve or reject the bugs reported, but at the same time, it has a right to change the state of the bug report and give rewards to the reporters. Admin role within the system can be considered as a platform moderator because he needs to approve companies, monitor bug activities, manage users' accounts.

3. User Interface Design

The user interface of the system has been designed in such a manner that it provides the user with an easy-to-use, simple, and intuitive interface. Navigability, readability of information, and interaction with the system features have been taken into account. The UI elements have been designed in conjunction with user stories and workflow models.

3.1 Registration and Login Interface

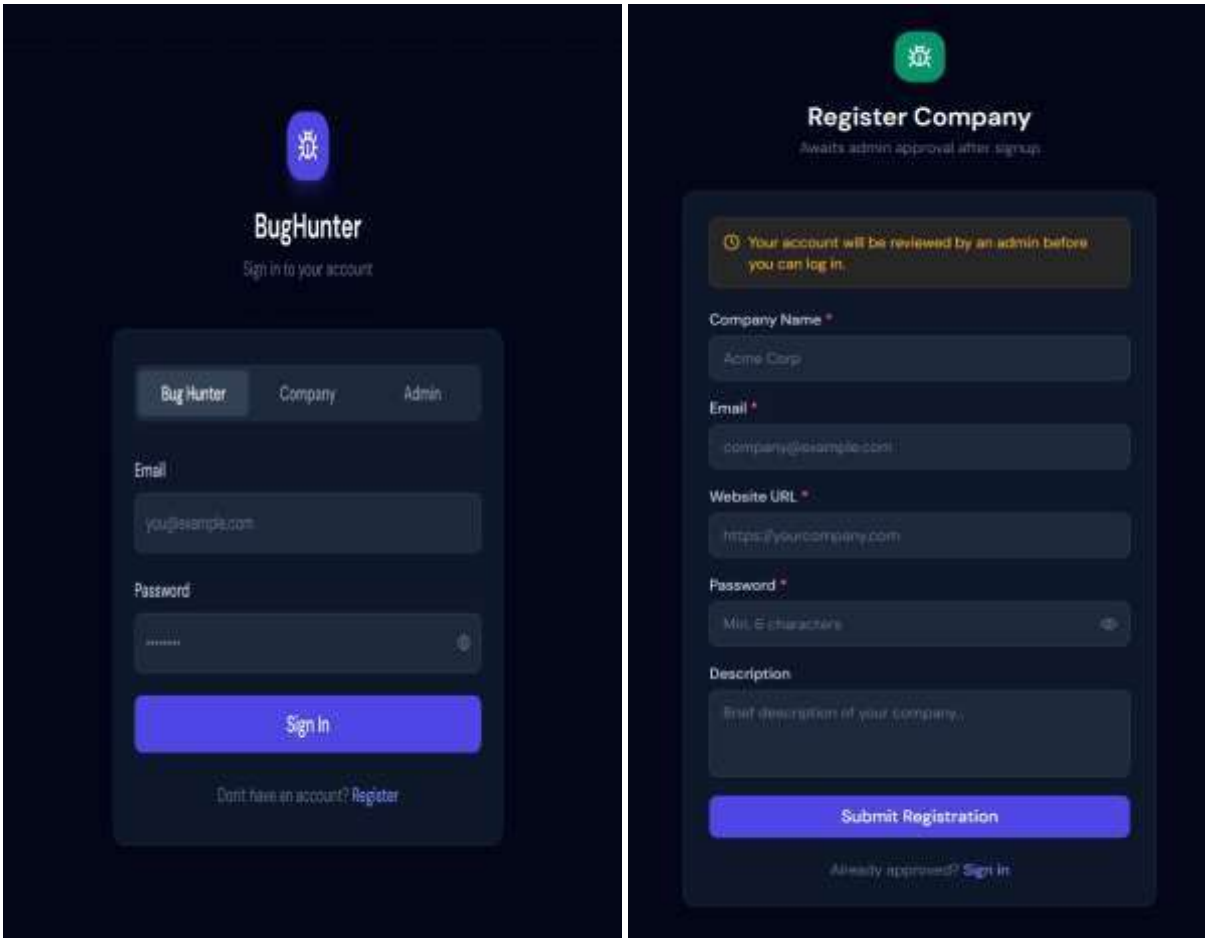


Fig 2: login and register page

This interface allows users as well as companies to create and log into their accounts securely. There is an area where credentials can be entered alongside the corresponding validation process.

3.2 User Dashboard

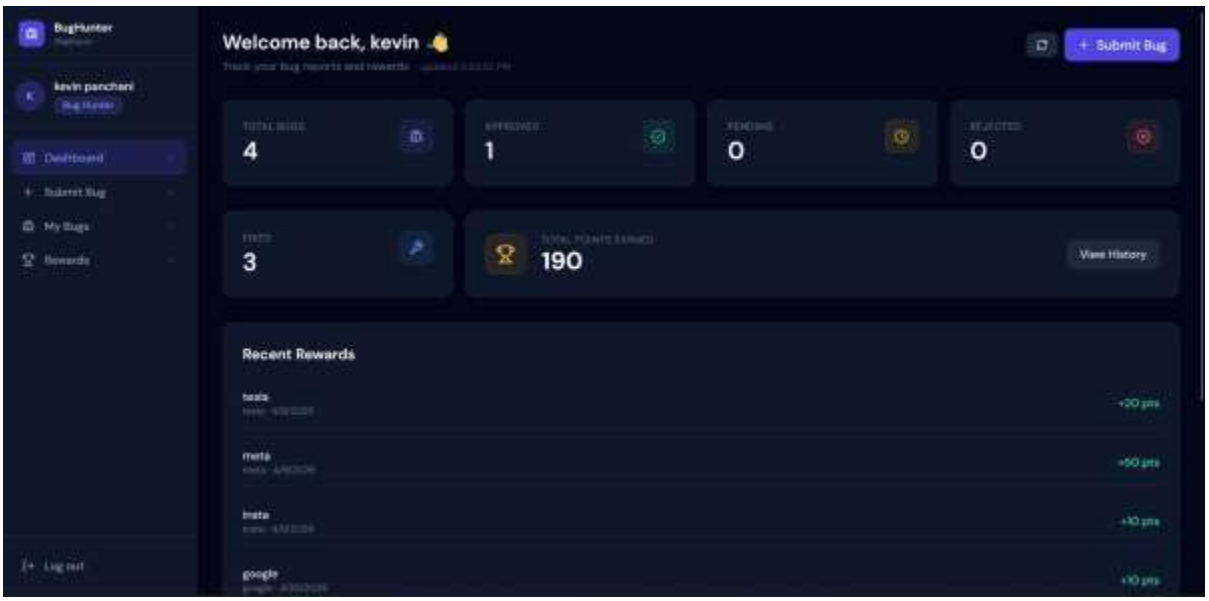


Fig 3: user dashboard

In the user dashboard user can see the how many bugs they submitted and approved along with the provided rewards. User can also see the how many approved bugs are fixed. Reward point history and section for the submit bug from where user can submit their bug.

3.3 submit Bug

When user click on the submit bug button this type of UI is open in front of user. User can enter the details like website name, URL, description, category and the proof as a video and photo. After that user can submit that bug and that bug shows into company dashboard.

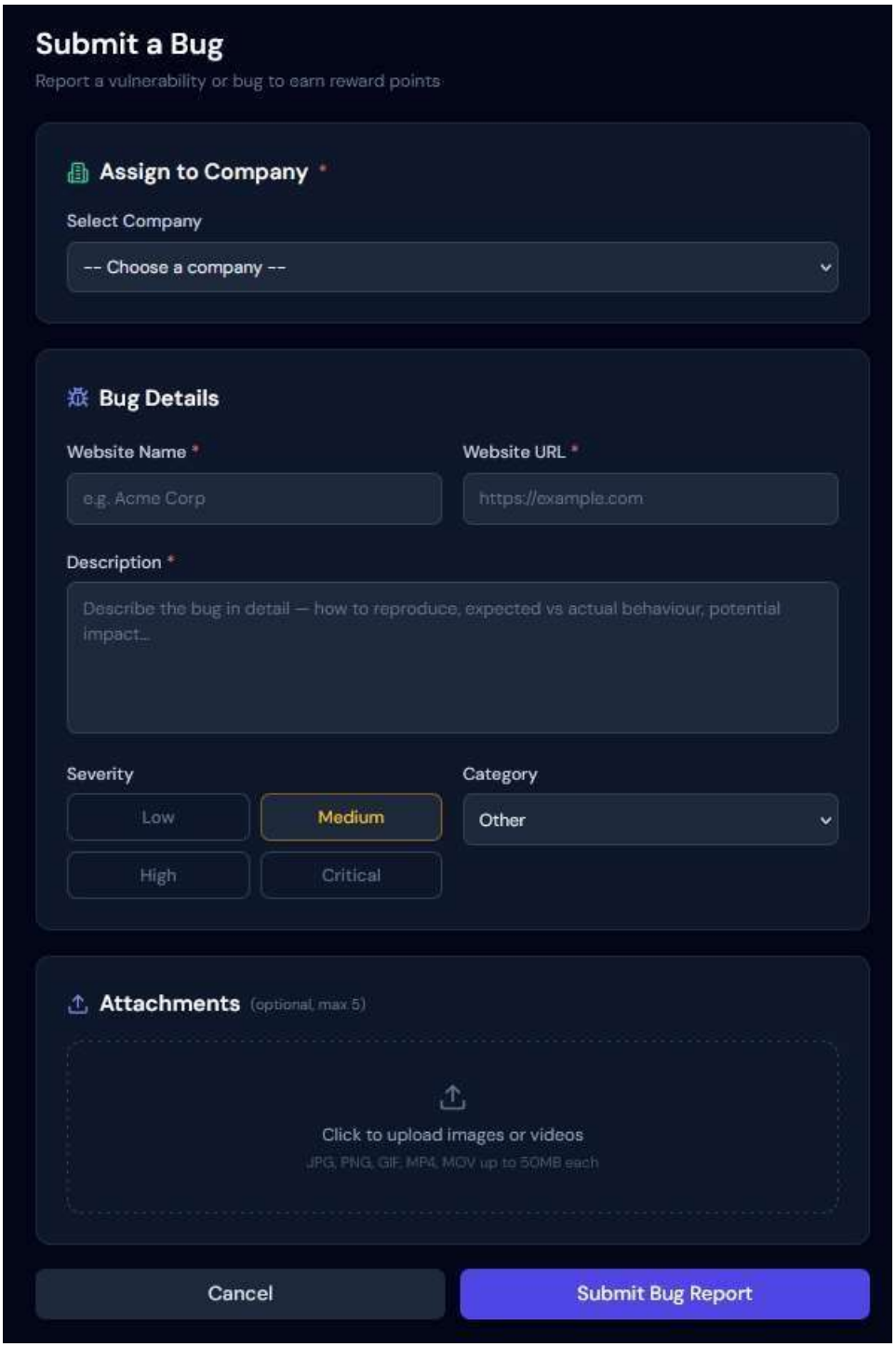


Fig 4: submit bug

3.4 Company Dashboard

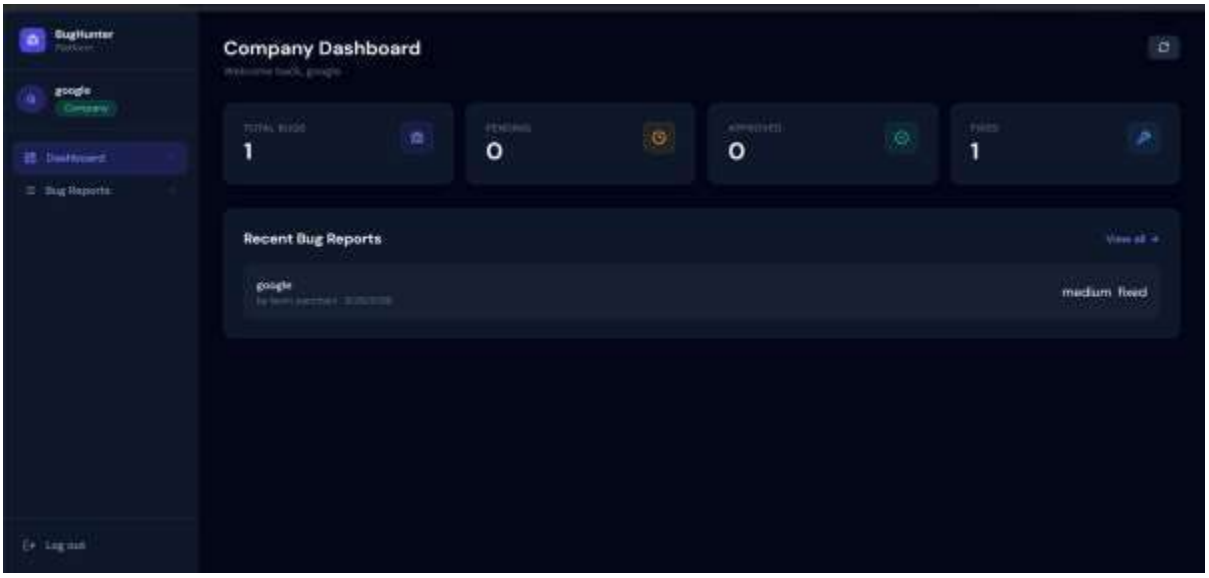


Fig 5: Company dashboard

It lists all submitted bug reports for the respective website. There is information provided for every report as well as its attachments. Also shows the how many bugs are pending, approved and fixed. Also at a bug approve time company can assign reward point to user.

3.5 Bug Review and Decision Interface

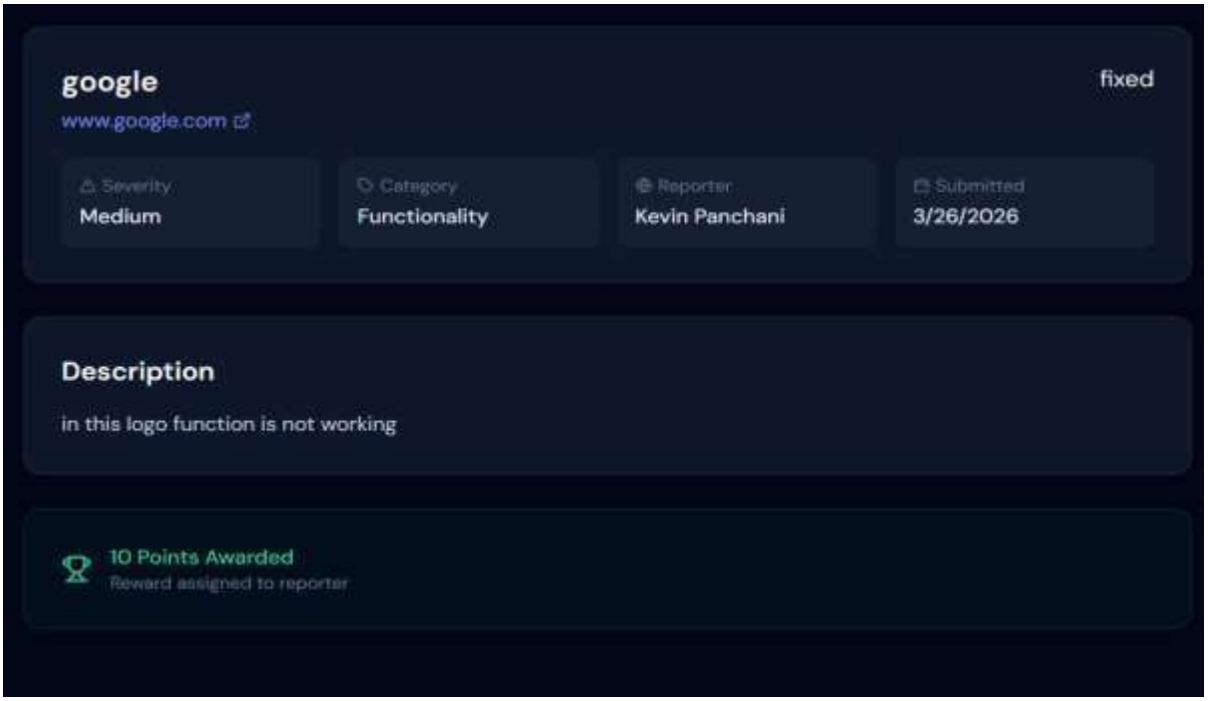
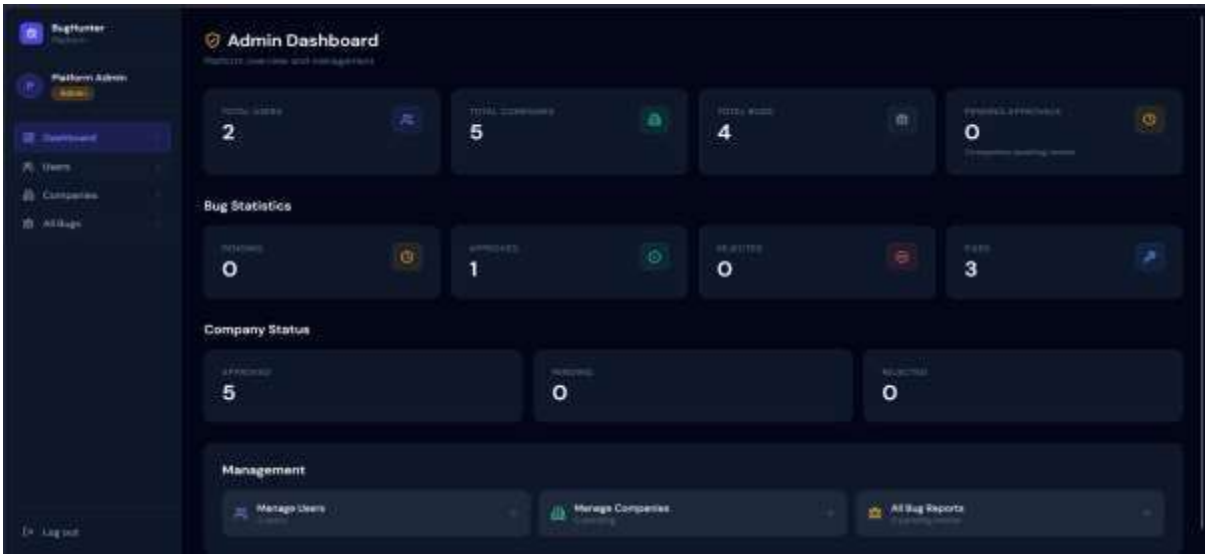
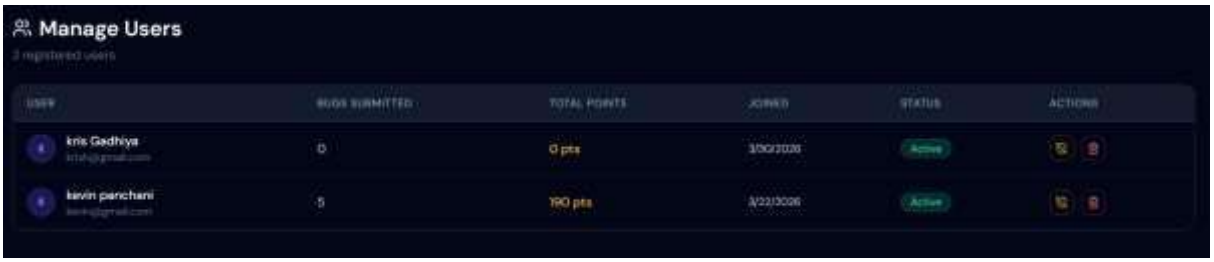


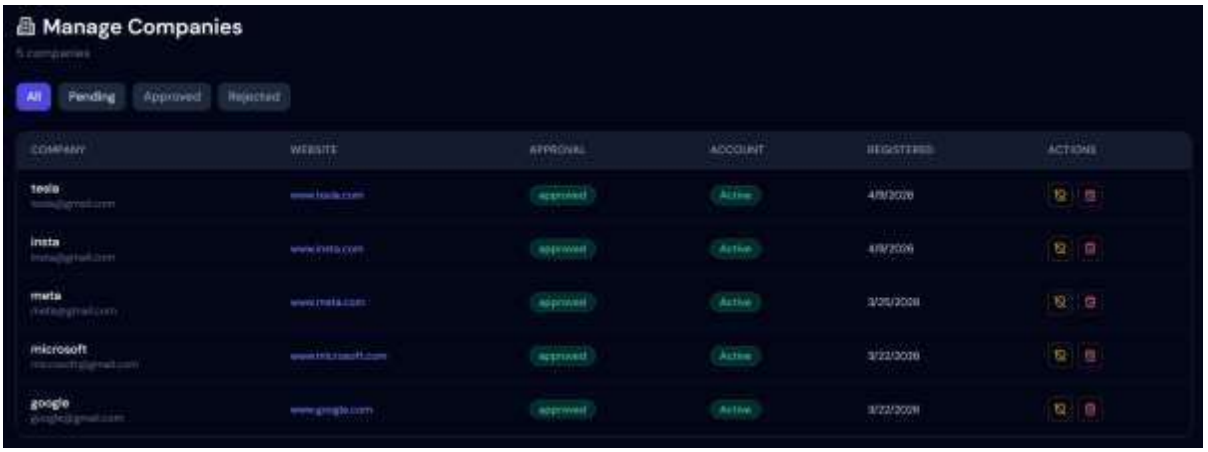
Fig 6: bug review

This interface gives companies the ability to examine individual bug reports, approve, reject or fix them, as well as providing feedback and reward points.

3.6 Admin Dashboard







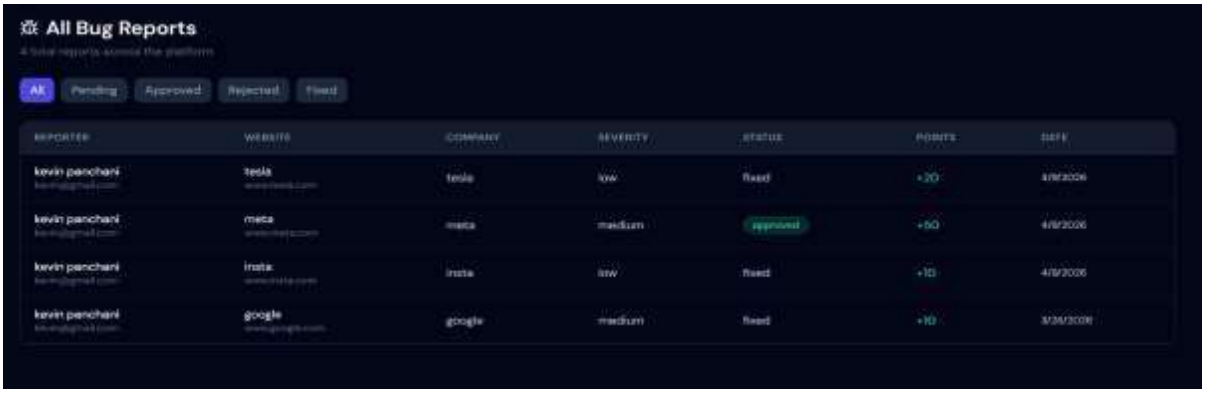


Fig 7: admin dashboard

This is the heart of the system like in the admin all things are covered. In the first part it shows, total company, total user and bugs and if any company approve request come then it also shows there.

In the second part it shows bug statistics like how many bug is approved and rejected and fixed so admin can get idea about its platform activity.

Next part shows the company status approved, rejected or pending. And in the management section it shows in the deep about all thing. If we show about mange user, then it shows total user and reward point they get and status of it.

Same for the company it shows all list of the company and its status admin can handle all thing from its dashboard.

In the bug management also admin can see the total bug, it’s status reward point, data of submission, status and company name.

4. Code Architecture and Implementation Structure

4.1 Tools and language used

Language

- JavaScript – Commonly used in both front and back end for consistency and efficiency

Frontend

- React 18 – Used to create dynamic and modular interfaces
- Vite – For quick development and deployment processes
- Tailwind CSS – Used to style interfaces responsively and in utility-first fashion
- Axios – To manage requests between the front and back ends
- React Context API – For state management

Backend

- Node.js – Runtime environment for server-side applications
- Express.js – Framework to create RESTful APIs
- MongoDB – NoSQL database to store application data
- Mongoose – Object Data Modeling (ODM) library for schema creation and validation

Authentication & Authorization

- JWT (jsonwebtoken) – Tokenization and user authentication
- bcryptjs – Password encryption and verification

File Management

- Multer – For managing file uploads and storage
- Cloudinary – Cloud service for storing images and videos

4.2 Reasons for Choosing These Tools

MERN stack was used due to the advantages provided by a JavaScript ecosystem, making the development process simpler due to the possibility of using the same programming language on all the levels of application development.

React was used as the frontend framework as it is highly flexible, possesses many useful features, is widely supported and has a big community. In comparison with Create React App, Vite has a smaller size and provides improved performance in development mode. In contrast with Bootstrap, Tailwind CSS does not require a large CSS file, giving more freedom in styling components.

Node.js and Express.js was chosen due to its simple, scalable and asynchronous architecture that allows processing multiple requests from different clients at once. The database used here is MongoDB as it has a flexible schema structure which can be updated in the future.

As opposed to other cloud hosting providers such as AWS S3, Cloudinary was chosen due to its native support of media optimization and convenient integration.

4.3 Code Structure and Workflow Models

The code is architected with a full-stack modular structure that comprises the frontend (React) and the backend (Node.js & Express) with distinct roles for both. The system's parts are classified into different folders like models, controllers, routes, components, and pages, among others, where each module corresponds to particular user stories of the system.

❖ **Backend Structure**

The backend is responsible for handling business logic, APIs, and database management. The following structure describes the backend architecture.

server.js

- The main entry file where the Express server, middleware, and routes are initialized. Supports the flow of activities of the application.

models/ (Database Schemas)

- User.js → Contains users' information and their reward points (satisfies US-01 and US-04)
- Company.js → Holds companies' information and approval status (satisfies US-05 and US-09)
- Bug.js → Contains bug reports, their statuses, and their severities (satisfies US-02 and US-07)
- Reward.js → Manages transactions (satisfies US-04 and US-08)

controllers/ (Business Logic)

- authController.js → Handles users' registration and login activities (satisfies US-01 and US-05)
- bugController.js → Handles submitting and updating bugs' statuses (satisfies US-02, US-03, and US-07)
- companyController.js → Handles reviewing and assigning rewards for the bugs (satisfies US-06 and US-08)
- adminController.js → Handles admin-related functions such as approving bugs, monitoring submissions, etc. (satisfies US-09, US-10, US-11, and US-12)

routes/ (API Routes)

- Routes such as /register, /login, /bugs, and so on; they map the frontend actions to the corresponding backend controllers, thereby forming the workflow steps of our application

middlewares/

- JWT-based authentication and authorization
- Role-based permissions (User, Company, and Admin)

❖ **Frontend Structure**

The frontend consists of a React-based Single Page Application (SPA), which provides user interfaces for all roles.

App.jsx

- Contains routing for various pages including the user, company, and admin dashboard.

context/AuthContext.jsx

- Maintains the global authentication context for the entire application.

components/

- Includes reusable UI components like protected routes and layout components used throughout all workflows.

pages/ (Role-Based UI)

- User Pages:
  - RegisterUser.jsx, LoginPage.jsx -> (US-01)
  - SubmitBug.jsx -> (US-02)
  - MyBugs.jsx, UserDashboard.jsx -> (US-03)
  - Rewards.jsx -> (US-04)

- Company Pages:
  - RegisterCompany.jsx -> (US-05)
  - CompanyBugList.jsx -> (US-06)

    - CompanyBugDetail.jsx -> (US-07, US-08)
- Admin Pages:
    - AdminCompanies.jsx -> (US-09)
    - AdminBugs.jsx -> (US-10)
    - AdminUser.jsx -> (US-11)
    - AdminDashboard.jsx -> (US-12)

❖ **Relationship to User Stories and Workflows**

- **User Workflow (US-01 - US-04)**

  → Register → Submit Bug → Check Progress → Earn Rewards

- **Company Workflow (US-05 - US-08)**

  → Register → Approve Company → Investigate Bugs → Approve/Reject Bugs → Reward

- **Admin Workflow (US-09 - US-12)**

  → Approve Companies → Monitor Bugs → Manage Users → See Stats

5. Implementation, Testing and Validation Strategy

The project was implemented via the Agile Scrum methodology with several sprints during which particular user stories were realized. Some of them included authentication, bug submission, company workflow realization, and administrator management. Each iteration of the software involved thorough testing.

The development team consisted of specialists responsible for frontend, backend, and testing that allowed efficient work with different user roles' needs (user, company, and admin).

System Testing Strategy

- Unit testing included checking components, controllers, and models (authentication, bug submission, etc.) for proper work.
- Integration testing included interactions between system modules (bug submission, company workflow, rewards).
- End-to-end testing entailed simulating workflows (registration of user, submitting bugs, their consideration, and rewarding).
- Performance and security testing was performed for a quick reaction time, secure token authentication, and safety from attacks.

System Validation

- User included submitting bugs, following their status, and receiving rewards.
- Company involved reviewing submitted bugs, accepting or declining them, and assigning rewards to the users.
- Administrator included approving companies, managing users, and monitoring overall system operation.

6. Video Description

The uploaded video illustrates the core functionality of the Bug Reporting Platform as it includes key user stories for all the roles – Admin, Company, and Bug Reporter (User).

Firstly, the video provides information about the admin's tasks, which start from the admin logging in and approving or rejecting companies' registration requests (US-09). Then, there is the process of monitoring of all the bug reports on the platform (US-10). Besides, there is information on how to manage users and companies via dashboard (US-11, US-12).

Secondly, the video depicts a company workflow. A company registers and then awaits an admin's decision (US-05). Once approved, the company logs in and reviews bug reports (US-06). Further, it shows how to approve or reject a bug report (US-07) and assign reward points depending on its level of severity (US-08).

Finally, the video shows how the workflow of a user looks like. The first task of the user is to register and log in (US-01). Then the user submits bug reports along with required information and files (US-02) and checks the status of their submission (US-03).

This sequence exemplifies a full workflow from end to end, which is in agreement with the testing approach mentioned above. This ensures that the entire application works as expected, with all the functionalities being implemented properly.